DESSERT 2026

Proteus

Constructing Semantically Equivalent Program Clones using LLMs and Execution Traces

Benita Magdalene Kingsley Prince · Ioannis Agadakos

School of Computing, Montclair State University



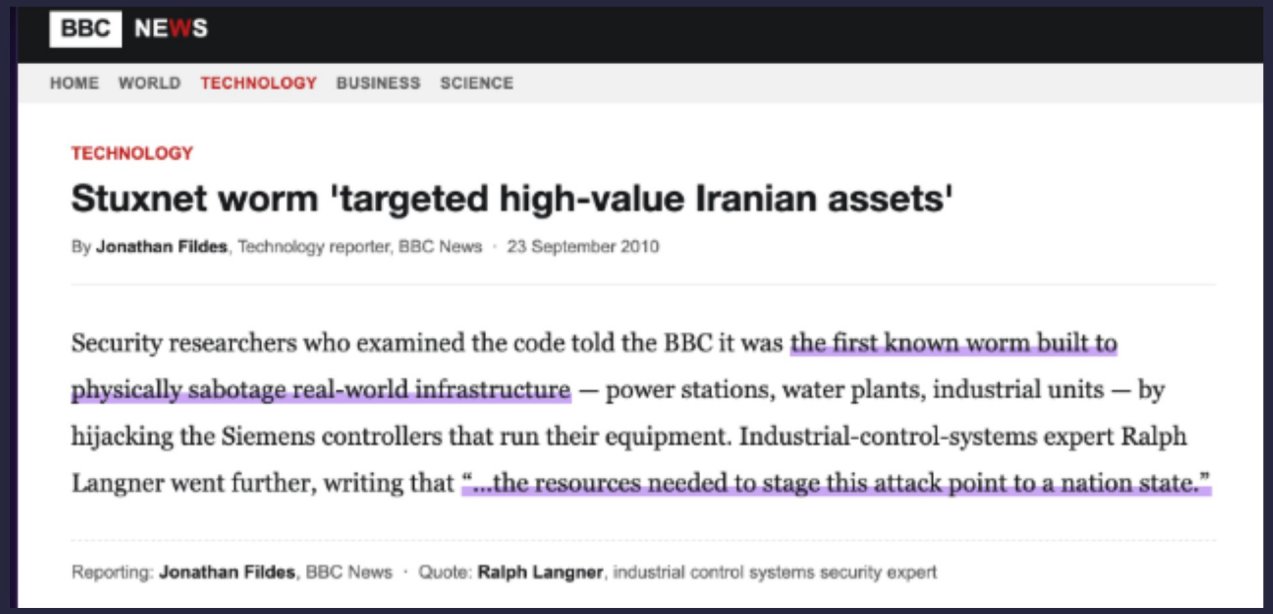
What is an Industrial Control System?

Software that estimates physical measurements and controls real infrastructure: sensor pipelines, pumps, boilers, telemetry gateways. It does not just serve data — it drives a physical process.

THE REQUIREMENT

The new code must behave exactly like the old one.

WHY THIS MATTERS TODAY



BBC NEWS

HOME WORLD TECHNOLOGY BUSINESS SCIENCE

TECHNOLOGY

Stuxnet worm 'targeted high-value Iranian assets'

By **Jonathan Fildes**, Technology reporter, BBC News · 23 September 2010

Security researchers who examined the code told the BBC it was the first known worm built to physically sabotage real-world infrastructure — power stations, water plants, industrial units — by hijacking the Siemens controllers that run their equipment. Industrial-control-systems expert Ralph Langner went further, writing that "...the resources needed to stage this attack point to a nation state."

Reporting: **Jonathan Fildes**, BBC News · Quote: **Ralph Langner**, industrial control systems security expert



CBS NEWS

HOME U.S. TECHNOLOGY WORLD POLITICS

TECHNOLOGY · CNET

Report: Stuxnet delivered to Iranian nuclear plant on thumb drive

By **Daniel Terdiman** · April 13, 2012, 6:51 AM EDT · CNET

Liam O'Murchu, who led Symantec's investigation into the worm, told CNET the infections reached industrial control companies across Iran, India and the U.S. energy sector, though he would not say how many or name any of them. "This is quite a serious development in the threat landscape," he said, warning that it handed an attacker direct control over a physical industrial system.

Reporting: **Daniel Terdiman**, CNET / CBS News · Quote: **Liam O'Murchu**, Symantec Security Response

The problem, concretely

Replace a dependency, a vulnerable, slow, or unmaintained library, without changing what the program does and alter the code logic.

BEFORE · STDLIB ZLIB

```
import zlib

def crc32(payload):
  return zlib.crc32(payload)
    & 0xFFFFFFFF
```

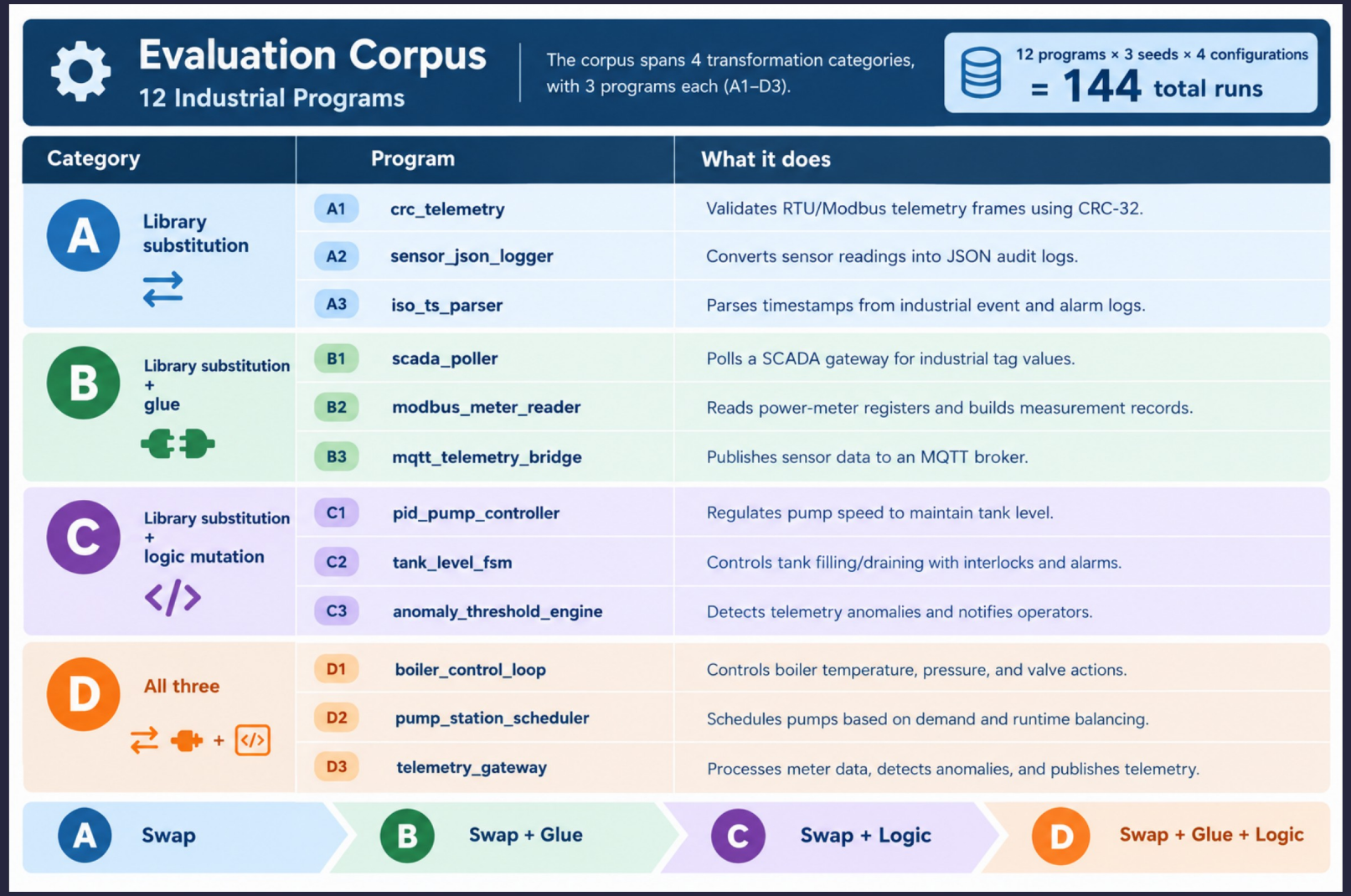
AFTER · FASTCRC

```
import fastcrc

def crc32(payload):
  return fastcrc.crc32.iso_hdlc(
    payload) & 0xFFFFFFFF
```

Same output on every input we tested. Verified directly against both libraries, not assumed.

12 Programs, 4 Categories



Industrial measurement & control domain · every program ships its own test suite and reference traces

CHECK 1 · DETERMINISTIC

When is a migration a success?

Compiles & tests pass

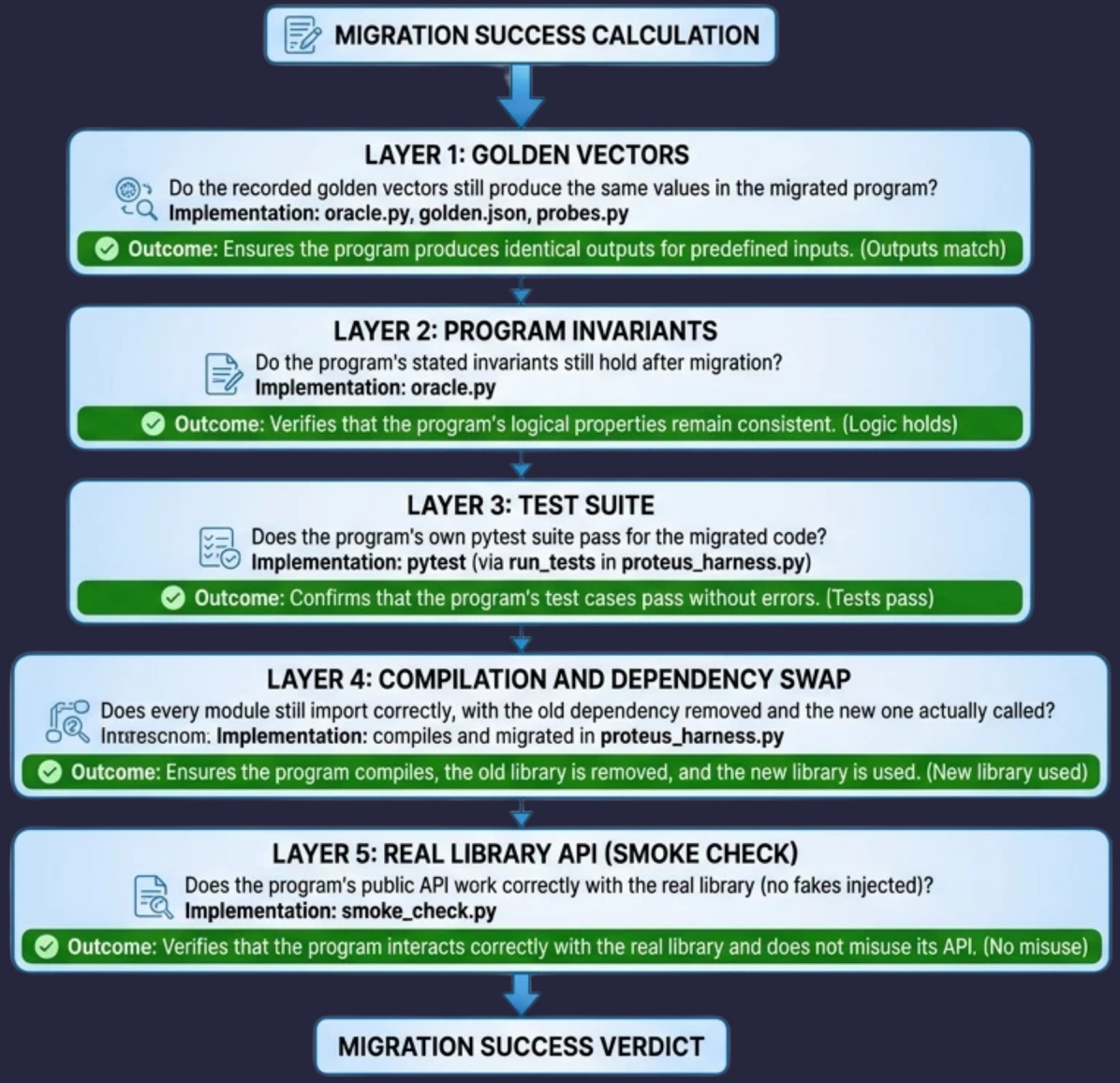
The clone runs and its full existing test suite passes.

Five-layer oracle

Imports, golden vectors, invariants, tests, and a live factory-execution smoke check against the real library.

A real migration

The source genuinely changed and the old dependency is verifiably gone.



CHECK 2 · LLM

The blind LLM judge



Every subject injects a fake client for testing, so the code that actually calls the new library often never runs under test. Deterministic checks cannot see that path. A second model reads the real source and judges behavior directly.

Clone, markers stripped → **claude-opus-5, temp 0** → **Reads behavior, not metadata** → **Verdict**

HOW IT'S RUN

Model: claude-opus-5, temperature 0

Different and stronger than the claude-sonnet-5 that wrote the clone — no model grades its own work, and the same clone gets the same verdict every time

Blind: every system-identifying marker renamed or stripped

The batch aborts if any marker survives — the reviewer never learns which configuration wrote a clone

Scope: behavior only

Style, structure, and library choice are explicitly excluded from review

WHAT IT PRODUCES

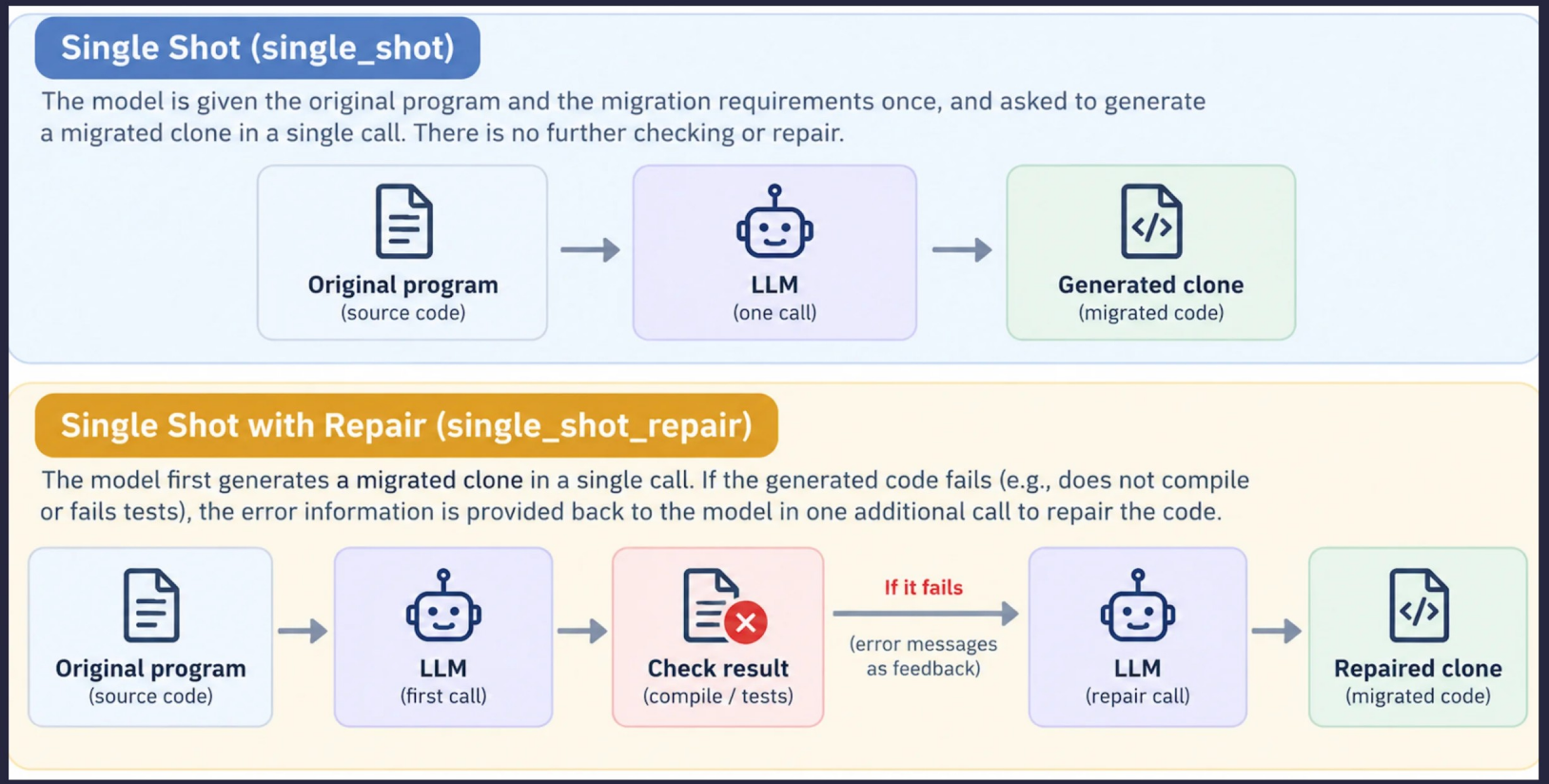
Findings — one per issue

file + the offending line, quoted + the runtime consequence, in one sentence

Worst finding wins. No findings → Clean.

BROKEN	fails on ordinary input
MAJOR	wrong on a case the original handled
MINOR	real difference, edge case only
CLEAN	no findings at all

Baselines



SINGLE-SHOT

One prompt. No agents, no tools, no verification loop.

50.0% migration success

18 of 36 runs

SINGLE-SHOT + REPAIR

One further call: the same failure evidence, fed back once.

55.6% migration success

20 of 36 runs

No agents. No multi-step pipeline. Just one prompt, and optionally one more.

One success, one failure

CLEAN · CRC_TELEMETRY

```
import fastcrc

def crc32(payload):
  return fastcrc.crc32.iso_hdlc(
    payload) & 0xFFFFFFFF
```

A one-line swap. Correct, no wrapper needed.

BROKEN · PID_PUMP_CONTROLLER

```
from pid import PID

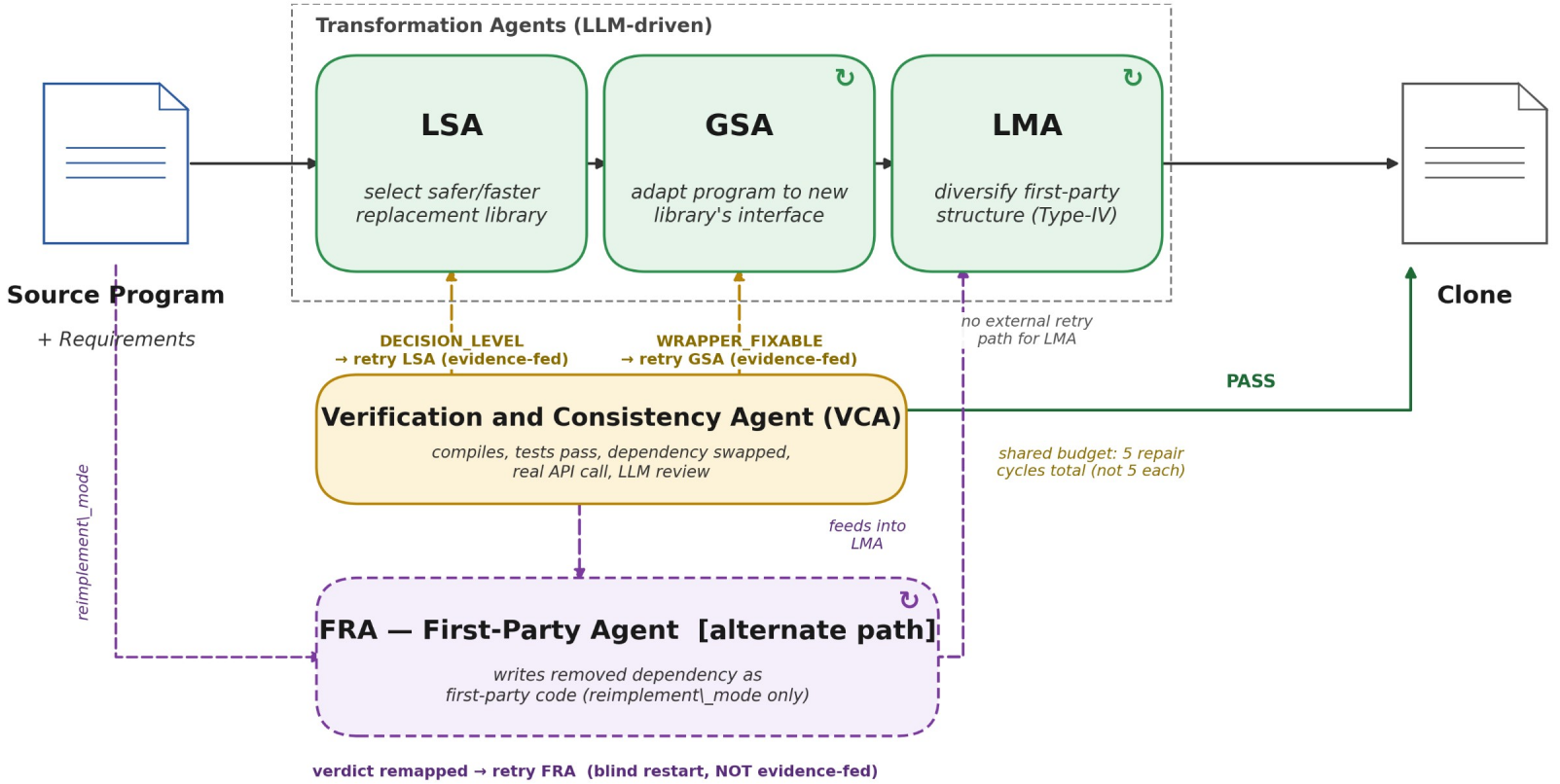
# pid.py was never written.
# ModuleNotFoundError on
# the very first import.
```

The model referenced a file it forgot to create.

Same model, same corpus, same seed budget. What differs is whether anything checks the output.

A multi-agent pipeline, not one prompt

Each agent owns one class of change and its own execute-and-repair loop; a verification agent gates the whole pipeline before anything ships.



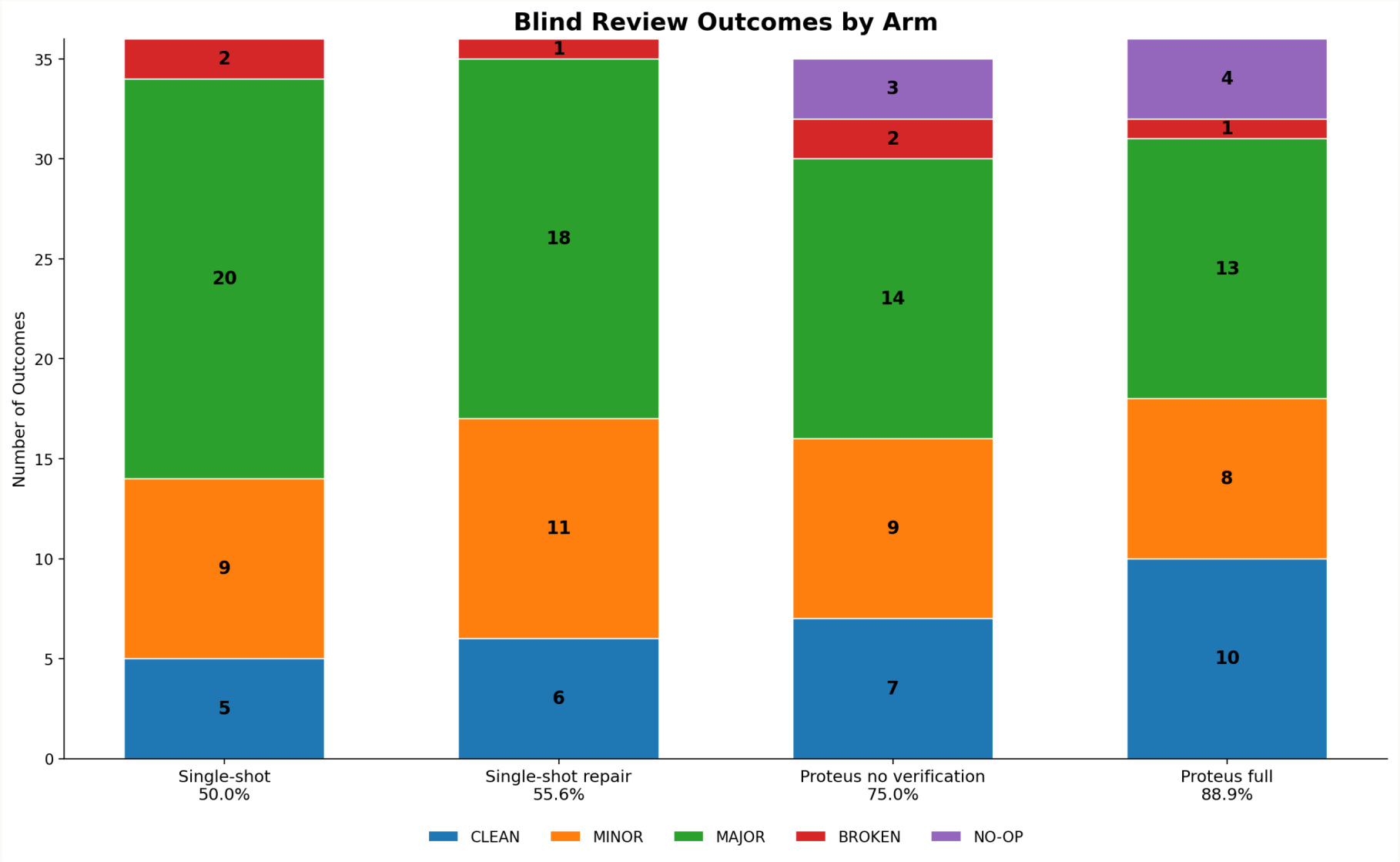
1. Local verification — execute-and-repair inside each agent's own attempt (LMA/FRA also check golden vectors here)
2. Global verification — VCA's deterministic acceptance gate, then independent LLM review

How Proteus differs

	Single-shot baseline	Proteus (full)
Median LLM calls per run	1	11
Agents	none	5, each with its own repair loop
Verification	No	Yes
Median cost per run	$0.03	~$0.31

More compute, spent on evidence-gathering and repair, not just a longer prompt

Final results, across all 144 clones



Does verification actually help?

FULL PIPELINE

75.0% → 88.9%

Proteus with verification off, vs. the same agents with it on

MINIMAL CASE

50.0% → 55.6%

One prompt, vs. the same prompt plus one evidence-fed repair call

Evidence-fed repair helps at both ends of the spectrum, agentic or not.

What Proteus gets right

Leads decisively on migration success

88.9% vs. 50.0% and 55.6% for both single-shot baselines

Perfect equivalence confirmation

36 of 36 runs, the only configuration that reaches every one

Most Clean, fewest Major or Broken

In absolute terms, despite accepting the most clones of any configuration

Auditable by design

Every substitution and mutation is attributable to the agent that made it

Where it still falls short

Not free

Roughly 33× the tokens, and 17× the dollar cost, of the plain baseline for the recall it buys

Verified is not perfect

The blind judge still finds real defects in clones that pass every deterministic check we run

One agent's retries are blind

The first-party agent restarts from scratch on rejection, with no evidence from the failure

Small sample size

The corpus being 12 program with 3 seeded experiment, does not give a huge statistical difference.

Future work

01 **Real PLC languages**
Retarget the agents at IEC 61131-3 — Structured Text, Ladder Logic — against real controller toolchains, not a Python stand-in

02 **An industrial control system simulator**
Check equivalence against modeled plant dynamics, not just recorded source-level I/O

03 **Attack surface, in both directions**
Does a real exploit still land on a diversified clone — and is the agent pipeline itself a new target

04 **A larger corpus**
Twelve subjects settle migration success; a bigger corpus, ideally real controller codebases, would settle the rest

Key takeaways

01 **Verification is what separates success from a plausible-looking failure**
Full \sys migrates 32 of 36 runs, far ahead of both single-shot baselines, because it catches what dependency-injected tests structurally cannot.

02 **Automated acceptance is necessary, not sufficient**
A blind, independent review still finds real defects in clones that pass every deterministic check we run.

03 **Detection and generation are two faces of the same problem**
Coupling them turns program equivalence from a question we answer into a property we can engineer — recorded, attributable, auditable.

Thank you for listening

Montclair State University · github.com/BenitaMags/Axiom

